

Section 1 — Technical Reference

System Architecture, Technology Decisions & Project Vision

Presenter: Ahmed Tawfik — Team Leader & System Architect

"We designed this system, made every architectural decision, and built the backend, frontend, database layer, AI integrations, Chrome extension, GNOME extension, and deployment pipeline as a cohesive engineering team. The seven sections of this project exist to cover the complete scope of our collaborative effort."

1. What Is Cortex?

Cortex is a **bilingual, AI-powered academic workspace** built specifically for Egyptian university students. It is not a note app with extra features, and it is not a collection of loosely integrated tools. It is a **unified platform** where five distinct tools — which students currently use in five separate, disconnected applications — exist as one coherent system that shares data, context, and intelligence.

The Five Systems Inside Cortex

1. Collaborative Notes A rich-text note editor where students write, organize, share, and publish study notes. Notes are built on Plate.js and stored as structured JSON — not plain text, not HTML. This JSON structure is what allows the backend to extract clean text for AI operations. Notes have:

- Nested folders and color-coded tags for organization
- Sharing with permission levels (viewer / editor)
- Public publishing — any note can become a community resource at a shareable URL, accessible without login
- Three AI tools in the sidebar: Summarize, Suggest Tags, and Library Assistant query
- Auto-save with a debounced 2-second timer (saves silently, never interrupts the user)
- Archive with restore, or permanent deletion

2. University Resource Registry (Data Catalog) A hierarchical catalog of every course in the student's university, organized as: `University → College → Major → Year Level → Course → Resources + Doctors`

Each course has resources (PDFs, links, videos) uploaded by admins and assigned doctors (professors). The catalog auto-filters to the student's curriculum based on their university profile. This replaces the WhatsApp group as the canonical source for academic materials.

3. Daily Planner with Calendar A per-day workspace structured with three sections: a rich-text note (same Plate.js editor), a task checklist (JSONB in the database, with optimistic UI toggle), and a highlight field (the day's best achievement). A calendar view renders a dot on every date where an entry exists. Every saved log runs an asynchronous embedding pipeline — the log's content is converted to a 768-dimensional vector and stored in the database, making every past day semantically searchable via the Library Assistant.

4. Habit Tracker, Pomodoro Timer & Social Leaderboard Three interlocking productivity systems:

- **Habits** — user-defined habits with frequency patterns (daily, weekly, monthly, custom). Streak calculation runs in the database using SQL window functions and CTEs. Breaking streaks is psychologically significant — this is the mechanism that makes habits stick.
- **Pomodoro Timer** — 25-minute focus / 5-minute break / 15-minute long break. Every completed session is logged to `pomodoro_sessions` with a timestamp, subject label, and source client (web, Chrome, or GNOME extension). This data feeds the leaderboard.
- **Friends & Social Leaderboard** — a friend request system and study groups. The weekly leaderboard is calculated by a PostgreSQL RPC function (`get_friends_leaderboard`) that aggregates total focus minutes across all clients since the last Monday. Resets every week.

5. AI Library Assistant The cross-cutting AI system. The student asks a question in natural language. The system converts the question to a 768-dimensional vector using Gemini's `text-embedding-004` model. `pgvector` finds the top-5 most semantically similar notes and daily logs in the student's library using cosine distance. Those documents are sent to Gemini Flash as context, along with the original question and a system prompt instructing it to answer only from the provided context. The response includes citations — which of the student's notes each part of the answer came from. This is RAG: Retrieval-Augmented Generation.

2. The Extensions: Cortex Outside the Browser

Beyond the web application, Cortex ships as two independent native integrations that connect to the same backend API.

Chrome Extension — Cortex Focus

A **Manifest V3 Chrome extension** with five tabs: Timer, Blocker, Social, History, Settings.

Timer tab: The Pomodoro timer runs in a background service worker. Chrome's `alarms` API is used to fire timer ticks at precise intervals even when the popup is closed and the browser is minimized. When a session completes, it is automatically synced to the Express backend via `API.recordSession()` with the user's stored JWT.

Blocker tab: Site blocking uses Chrome's `declarativeNetRequest` API — not a content script overlay, not JavaScript interception. Blocking rules are activated as `declarativeNetRequest` dynamic rules when a focus session begins, and removed when it ends. Blocked sites fail at the HTTP request level — the browser never even initiates the connection.

Authentication: The user signs in once on the web app. The JWT is stored in `chrome.storage.local`. The extension reads this token and includes it in every API request via the `Authorization: Bearer <token>` header.

GNOME Extension — Tamatem (`tamatem@frey.dev`)

A **GNOME Shell extension** (shell versions 48–50) written in GJS (GNOME JavaScript). It integrates the Pomodoro timer directly into the Linux desktop environment's system top bar.

Timer display: A `St.Label` widget in the GNOME panel indicator shows the remaining time and current subject. It updates on a GLib timer callback.

OS-level site blocking: Blocking is implemented at the network/hosts level — not through the browser. When a focus session starts, the extension modifies the system's DNS resolution for blocked domains. No browser process is required.

Session sync: Completed sessions are synced to the Cortex Express backend using GNOME Shell's `Soup.Session` HTTP library. The JWT is stored in GNOME's settings schema (`Gio.Settings`) and retrieved for each API call.

This makes Cortex one of the only academic platforms that extends its functionality not just to a browser extension, but to the operating system itself — an important

differentiator for CS students who live in a Linux terminal.

3. Why This System Exists

We did not start by picking technologies. We started by living the problem as students at Menoufia University.

Our academic life was spread across at minimum five disconnected tools:

- Study notes in Google Docs — no AI, no course structure, no connection to anything else
- Lecture materials in a WhatsApp group — expires, unsearchable after a few weeks, no organization
- A habit tracker on a phone that knows nothing about our curriculum or study schedule
- ChatGPT that answers from the internet but has never read our notes or our professor's slides
- No tool designed for Arabic-speaking university students in Egypt

Every tool was built for a generic user. None of them talked to each other. We spent more time managing tools than studying.

Cortex is our answer. A system built ground-up where every feature was designed to work with every other feature, in Arabic and English, from the first day of development.

4. The Core Architectural Decisions

Decision 1: The Frontend Never Touches the Database

The rule: `Client → Express Backend → Supabase`. Never `Client → Supabase directly`.

Supabase provides two database access modes:

- **Anon key** — safe for client use, respects RLS policies
- **Service role key** — bypasses ALL security policies; has full admin access to every table

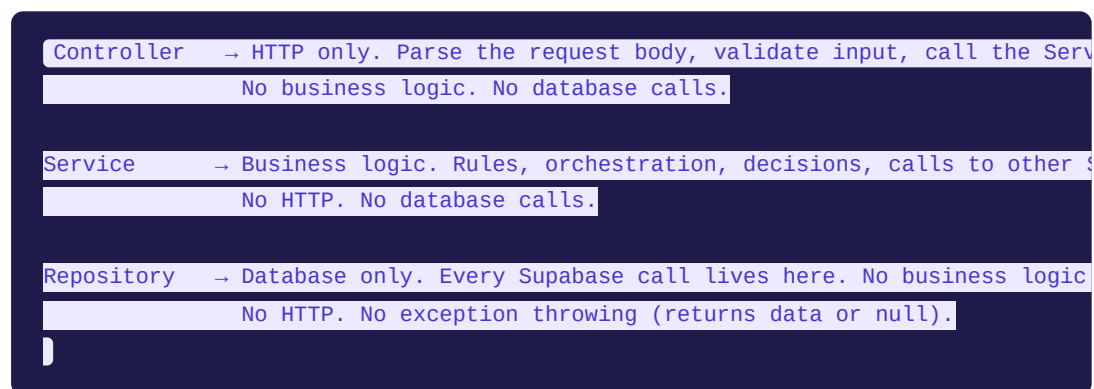
Many projects use the service role key from the frontend when they need to bypass RLS for admin operations. We explicitly refused this pattern.

Why this matters: The service role key lives exclusively on the Railway Express server. No browser, no extension, no client environment ever sees it. Even if an attacker fully controls the frontend JavaScript, they cannot reach the database directly. The only attack surface is the Express API — and the API enforces authentication and authorization on every endpoint.

Secondary benefit: Adding new clients is trivial. The Chrome extension and GNOME extension were added without any change to the database access model. They are simply additional callers of the same Express API. A future mobile app would follow the same pattern.

Decision 2: Three-Layer Backend (Controller → Service → Repository)

Every domain in the system follows an identical structure:



The constraint that enforces this: No Supabase import is allowed outside a Repository file. This is enforced by code review convention and is visually obvious when violated.

Why this matters in practice:

- Replacing the database requires changing only Repository files — controllers and services do not know what database is being used
- Business logic can be unit tested without a running database
- Adding a new feature means following a predictable, established pattern — new team members can onboard by reading one domain and understanding all seven

The seven backend domains: `Auth`, `Notes`, `Daily`, `Data`, `AI`, `Admin`, `Workspace`

Decision 3: Next.js 15 App Router with Server Components

The catalog problem: The university course catalog contains hundreds of courses. With a Single-Page Application (Vite, CRA), the browser downloads all catalog data and renders in JavaScript. The user sees a loading state. With Next.js Server Components,

the catalog renders server-side and arrives as finished HTML. Instant first paint. No JavaScript loading state.

This is not just a performance choice — it is a security choice for the admin panel. The admin layout calls `requireAdmin()` before any page component renders. This redirect happens on the server before a single byte of the admin page is sent to the client. There is no JavaScript code to bypass, no client-side guard to circumvent.

Caching strategy: The catalog pages use `next: { revalidate: 300 }` — they are cached for 5 minutes and revalidated in the background. This means the first student to load the page hits the database; the next 300 students get the cached response instantly. Perfect for content that changes infrequently but is read frequently.

Decision 4: Supabase for Database, Auth, and pgvector

Three specific capabilities drove this choice:

Row-Level Security (RLS): Policies are SQL expressions that execute at the PostgreSQL query level — not the application level. They are attached to every table.

Example policy on the `notes` table:

```
-- Users can only see notes they own or notes shared with them
CREATE POLICY "notes_select_policy" ON notes
FOR SELECT USING (
  user_id = auth.uid()
  OR id IN (
    SELECT note_id FROM note_shares WHERE user_id = auth.uid()
  )
  OR is_public = true
);
```

Even if an application bug constructs a query that would expose another user's notes — the database rejects it. RLS is defense in depth at the data layer.

pgvector: The PostgreSQL extension for vector similarity search. Notes and daily logs are embedded as `vector(768)` columns. The Library Assistant queries using cosine similarity:

```
SELECT id, title, 1 - (embedding <=> query_embedding) AS similarity
FROM notes
WHERE user_id = $1
```

```
ORDER BY embedding <=> query_embedding
LIMIT 5;
```

Setting this up on raw PostgreSQL would have required manual extension installation and configuration. Supabase enables it with one click in the dashboard.

Managed Auth: JWT issuance, refresh token rotation, `getUser()` server-side verification, and Next.js SSR cookie helpers — all without custom implementation. Supabase Auth handles the full JWT lifecycle.

Decision 5: Google Gemini

The AI subsystem required two capabilities simultaneously: a quality conversational model for the Library Assistant and editor AI, and an embedding model for vector search.

Evaluation across the three major providers:

Criterion	OpenAI	Google Gemini	Anthropic
Arabic quality	Good	Excellent	Good
Cost per 1M tokens	\$10–30	\$0.075	\$3–15
Embedding model	Yes	Yes (<code>text-embedding-004</code>)	No
Context window	128K	1M	200K

Two decisive factors: Arabic comprehension quality (tested with actual academic Arabic text from Egyptian university materials) and the 100–400× cost difference. For a student-facing platform where every AI call has a cost, Gemini's pricing makes sustained usage economically viable. Anthropic was eliminated immediately — no embedding model means no RAG pipeline.

Decision 6: Plate.js for the Rich Text Editor

The note editor must produce a structured, machine-readable document — not raw HTML — because the backend processes note content for AI features.

What Plate.js produces (JSON AST):

```
{
  "type": "doc",
  "children": [
```



```
{ "type": "h1", "children": [{ "text": "Operating Systems – Chapter 5" }],
  "type": "p", "children": [{ "text": "CPU scheduling determines..." }],
  "type": "code_block", "lang": "c", "children": [{ "text": "// Round Robin example" }],
}
```

What Quill produces:

```
<h1>Operating Systems – Chapter 5</h1>
<p>CPU scheduling determines...</p>
<pre><code class="ql-syntax">// Round Robin example</code></pre>
```

To extract plain text from Plate.js: walk the JSON tree recursively, extract all `text` leaf values. Clean, reliable, fast.

To extract plain text from Quill: parse HTML, strip tags, handle edge cases for every block type. Fragile and error-prone.

This choice was the foundation of every AI feature in the system.

Draft.js was eliminated because it is deprecated. **TipTap** was considered but its JSON schema is less structured than Plate's and its AI integration path was more complex.

Decision 7: Manifest V3 + `declarativeNetRequest` for Chrome Extension

Chrome deprecated Manifest V2 and required all extensions to use V3. The site blocker could have been implemented as a content script that injects CSS to hide page content — but this can be bypassed by the user, and it does not actually block network requests.

`declarativeNetRequest` blocks at the HTTP request level. The browser never initiates the connection to the blocked domain. This cannot be bypassed by the user, it works across all browser tabs simultaneously, and it is the only approach that Chrome supports in MV3.

Implementation:

```
// Activating site blocks when focus session starts
chrome.declarativeNetRequest.updateDynamicRules({
  addRules: blockedSites.map((site, index) => ({
    id: index + 1,
    priority: 1,
```

```
    action: { type: 'block' },
    condition: { urlFilter: site, resourceTypes: ['main_frame'] }
  )),
  removeRuleIds: []
});
```

Decision 8: GNOME Shell Extension for Linux Desktop Integration

Linux is the primary development environment for most computer science students. Ubuntu and Fedora both use GNOME as their default desktop environment. Building a GNOME Shell extension extends Cortex to where these students already are — in their terminal, in their desktop environment — without requiring a browser tab to be open.

The GNOME extension shares no code with the web app or Chrome extension. It is written in GJS (GNOME JavaScript), uses GNOME Shell APIs (`St`, `GLib`, `Gio`, `Soup`), and communicates with the Cortex Express backend over HTTP. It is a completely independent client.

Decision 9: next-intl for First-Class Arabic/English Bilingualism

Bilingual support was a day-one requirement, not an afterthought. The architectural implications:

Translation files: Every UI string lives in `messages/en.json` or `messages/ar.json`. No hardcoded English strings anywhere in the codebase. Server components use `getTranslations()`. Client components use the `useTranslations()` hook.

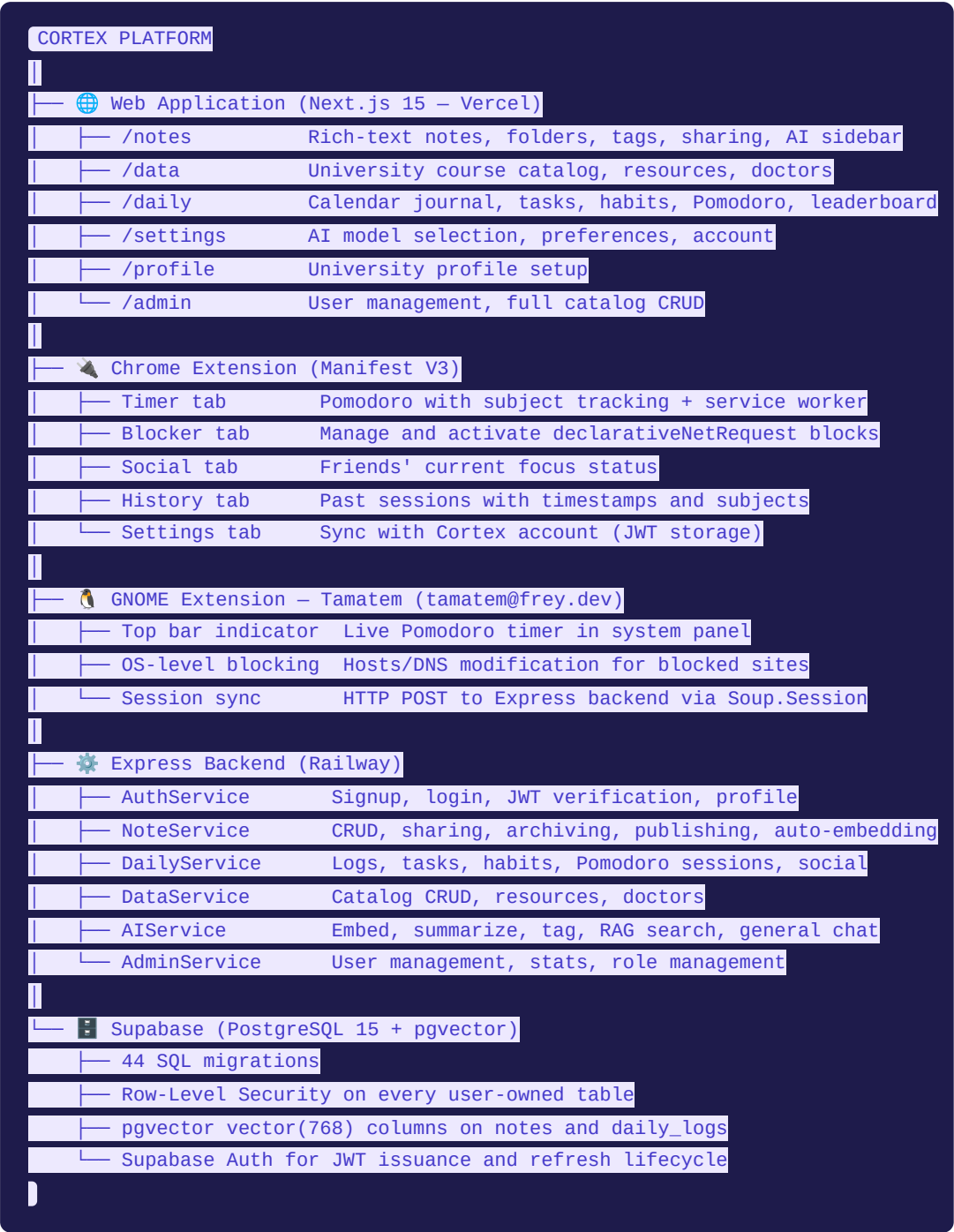
RTL layout: The entire layout uses CSS logical properties:

- `padding-inline-start` instead of `padding-left`
- `margin-inline-end` instead of `margin-right`
- `text-start` instead of `text-left`

When `dir="rtl"` is set on the HTML element (triggered by selecting Arabic), CSS logical properties automatically mirror the layout — right becomes left, start becomes right. Arabic-specific CSS overrides are not needed.

Database: Every catalog entity has both `name_en` and `name_ar` columns. When displaying a university or course name, the frontend reads the appropriate column based on the current locale. Content is bilingual, not just the interface.

5. The Full System Map



6. Database Schema Overview

Core Tables

profiles — extends Supabase Auth users

```
id uuid PRIMARY KEY REFERENCES auth.users(id),
name text,
role text DEFAULT 'user', -- 'user' | 'admin'
university_id uuid REFERENCES universities(id),
college_id uuid REFERENCES colleges(id),
major_id uuid REFERENCES majors(id),
year_level int
```

Created automatically by a database trigger when a user registers. The trigger fires on **INSERT** into **auth.users** and calls a stored procedure to create the corresponding profile row.

notes — the primary note document

```
id uuid PRIMARY KEY,
user_id uuid REFERENCES profiles(id),
title text,
content jsonb, -- Plate.js JSON AST
content_text text, -- extracted plain text for AI
embedding vector(768), -- pgvector, null until embedded
is_public boolean DEFAULT false,
is_archived boolean DEFAULT false,
folder_id uuid REFERENCES folders(id),
created_at timestampz,
updated_at timestampz
```

daily_logs — one row per user per day

```
id uuid PRIMARY KEY,
user_id uuid REFERENCES profiles(id),
log_date date,
content jsonb, -- Plate.js JSON AST
content_text text,
highlight text,
tasks jsonb, -- [{id, text, completed, order}]
embedding vector(768), -- null until syncLogEmbedding runs
UNIQUE(user_id, log_date)
```

pomodoro_sessions — one row per completed focus session

```

id uuid PRIMARY KEY,
user_id uuid REFERENCES profiles(id),
subject text,
duration_minutes int,
source text,          -- 'web' | 'chrome' | 'gnome'
completed_at timestampz

```

habits + **habit_logs** — habits with frequency tracking

```

-- habits
id uuid, user_id uuid, name text, emoji text,
frequency text,          -- 'daily' | 'weekly' | 'monthly' | 'custom'
target_days int[],       -- [1,2,3,4,5] for weekdays-only habits
color text

-- habit_logs
id uuid, habit_id uuid, user_id uuid, log_date date

```

note_shares — sharing with permission levels

```

id uuid, note_id uuid, user_id uuid,
permission text          -- 'viewer' | 'editor'

```

friendships — bidirectional social graph

```

id uuid,
requester_id uuid REFERENCES profiles(id),
addressee_id uuid REFERENCES profiles(id),
status text              -- 'pending' | 'accepted' | 'blocked'

```

Catalog Tables

Five tables in a strict hierarchy with foreign key constraints enforcing parent-child relationships:

```

universities (id, name_en, name_ar)
├── colleges (id, university_id, name_en, name_ar)
│   ├── majors (id, college_id, name_en, name_ar)

```

```

└─ year_levels (id, major_id, level int)
└─ courses (id, year_level_id, code, name_en, name_ar, created_at)
└─ resources (id, course_id, title, type, url)
└─ doctors (id, name_en, name_ar, specialty)
└─ course_doctors (course_id, doctor_id) -- join table

```

7. API Architecture

Authentication Pattern

Every protected route passes through the `authenticate` middleware:

```

// middleware/authenticate.ts
export async function authenticate(req: Request, res: Response, next: NextFunction) {
  const token = req.headers.authorization?.replace('Bearer ', '');
  if (!token) return res.status(401).json({ error: 'No token' });

  // supabase.auth.getUser() – server-to-server verification
  // This is NOT just reading the token locally; it verifies with Supabase's servers
  const { data: { user }, error } = await supabase.auth.getUser(token);
  if (error || !user) return res.status(401).json({ error: 'Invalid token' });

  req.user = user;
  next();
}

```

The critical detail: `getUser()` makes a network call to Supabase's servers to verify the JWT cryptographically AND confirm the token has not been revoked. This is more secure than verifying the JWT signature locally (which is what `getSession()` does).

Admin Middleware

Routes that require admin access have an additional middleware layer:

```

// middleware/requireAdmin.ts
export async function requireAdmin(req: Request, res: Response, next: NextFunction) {
  const profile = await ProfileRepository.getById(req.user.id);
  if (profile?.role !== 'admin') {
    return res.status(403).json({ error: 'Admin access required' });
  }
  next();
}

```

```
    next();  
  }
```

API Endpoint Overview

Domain	Key Endpoints
Auth	<code>POST /auth/login</code> , <code>POST /auth/register</code> , <code>POST /auth/logout</code> , <code>GET /auth/profile</code> , <code>PUT /auth/profile</code>
Notes	<code>GET /notes</code> , <code>POST /notes</code> , <code>PUT /notes/:id</code> , <code>DELETE /notes/:id</code> , <code>POST /notes/:id/share</code> , <code>POST /notes/:id/archive</code> , <code>PATCH /notes/:id/publish</code>
Daily	<code>GET /daily/:date</code> , <code>PUT /daily/:date</code> , <code>POST /daily/:date/tasks</code> , <code>PATCH /daily/tasks/:id</code> , <code>GET /daily/calendar/:month</code>
Habits	<code>GET /habits</code> , <code>POST /habits</code> , <code>POST /habits/:id/log</code> , <code>DELETE /habits/:id/log/:date</code>
Pomodoro	<code>POST /pomodoro/sessions</code> , <code>GET /pomodoro/sessions</code> , <code>GET /pomodoro/leaderboard</code>
Data	<code>GET /data/universities</code> , <code>GET /data/courses/:id</code> , <code>POST /data/courses</code> (admin), <code>POST /data/resources</code> (admin)
AI	<code>POST /ai/embed/:noteId</code> , <code>POST /ai/summarize/:noteId</code> , <code>POST /ai/tags/:noteId</code> , <code>POST /ai/library</code> , <code>POST /ai/chat</code>
Admin	<code>GET /admin/users</code> , <code>PUT /admin/users/:id/role</code> , <code>GET /admin/stats</code>

8. The AI Architecture in Detail

Three AI Layers

Layer 1 — Editor AI (inline, streaming) Triggered by the `/ai` slash command or `Ctrl+J` keyboard shortcut in the Plate.js editor. Uses the Vercel AI SDK for streaming. Before generating, a quick `generateObject()` call classifies the user's intent:

```
const { object: intent } = await generateObject({  
  model: selectedModel,  
  schema: z.object({  
    action: z.enum(['generate', 'edit', 'comment']),
```

```

    hasSelection: z.boolean()
  }},
  prompt: `User prompt: "${userPrompt}". Selected text: "${selectedText} ||
  }));

```

Based on the intent, the system prompt changes: generate = write from scratch; edit = transform selected text; comment = annotate without replacing. The response streams token-by-token using `streamText()` and is inserted character-by-character into the Plate.js document via the editor's `insertText` transform.

Layer 2 — Note AI Tools (sidebar) Three operations that run on a per-note basis:

- `summarizeNote()` — extractive algorithm: TF-IDF scoring on sentences, top-5 sentences selected and returned in original order. No external API call. Runs in milliseconds.
- `suggestTags()` — word frequency analysis: tokenize text, filter stopwords, count term frequency, return top-N terms as suggested tags. Also local, no API call.
- `embedNote()` — calls `text-embedding-004` with the note's `content_text`, stores the resulting `vector(768)` in the database. Updates the `embedding` column on the note row.

Layer 3 — Library Assistant (RAG) The full pipeline:

```

1. User asks: "How does Round Robin scheduling work?"
↓
2. Embed the question:
  GET text-embedding-004("How does Round Robin scheduling work?")
  → [0.023, -0.441, 0.117, ...] (768 floats)
↓
3. Vector search in pgvector:
  SELECT id, title, content_text,
         1 - (embedding <=> $query_vector) AS similarity
  FROM notes
  WHERE user_id = $user_id
  AND embedding IS NOT NULL
  ORDER BY embedding <=> $query_vector
  LIMIT 5;
↓
4. Filter: discard results with similarity < 0.7 (tuned threshold)
↓
5. Build context string from the top-N results
↓
6. Send to Gemini Flash:
  System: "Answer using ONLY these notes. Cite which note each fact comes from

```



```
User: "How does Round Robin scheduling work?"
Context: [note 1 text] [note 2 text] [note 3 text]
↓
7. Stream grounded answer with source citations to the frontend
|
```

The same pipeline runs for daily log embeddings — after every save of a daily log, `syncLogEmbedding()` runs asynchronously, converting the log's `content_text` to a vector. Daily logs are included in the pgvector search alongside notes.

9. Security Architecture

Defense in Depth — Four Layers

Layer	Mechanism	What it prevents
1 — Transport	HTTPS everywhere (Vercel, Railway, Supabase all enforce TLS)	Network interception
2 — Authentication	JWT verified via <code>supabase.auth.getUser()</code> on every request	Unauthenticated access, forged tokens
3 — Authorization	Admin middleware, ownership checks in Service layer	Horizontal privilege escalation
4 — Database	RLS policies on every user-owned table	Data leakage even if layers 1–3 are compromised

The Service Role Key

The service role key is the single most sensitive credential in the system. It is stored as an environment variable on Railway (`SUPABASE_SERVICE_ROLE_KEY`). It never appears in:

- Vercel environment variables
- The Next.js frontend codebase
- The Chrome extension code
- The GNOME extension code
- Any client-side environment

Even an admin user on the Vercel dashboard would not find this key.

httpOnly Cookie Security

The JWT is stored in an httpOnly cookie on the web app. httpOnly cookies are invisible to JavaScript — `document.cookie` will not show them. An XSS attack that injects JavaScript into the page cannot steal the authentication token, because JavaScript cannot read the cookie.

Cookie attributes:

```
httpOnly: true    → invisible to JavaScript
secure: true      → HTTPS only (rejected over HTTP)
sameSite: 'lax'   → blocks cross-site request forgery in most cases
```

RLS Policy Examples

```
-- Notes: users see only their own notes, shared notes, or public notes
CREATE POLICY "notes_select" ON notes FOR SELECT
  USING (
    user_id = auth.uid()
    OR is_public = true
    OR id IN (SELECT note_id FROM note_shares WHERE user_id = auth.uid())
  );

-- Daily logs: users see only their own logs
CREATE POLICY "daily_logs_select" ON daily_logs FOR SELECT
  USING (user_id = auth.uid());

-- Courses: admins can insert, everyone else read-only
CREATE POLICY "courses_insert" ON courses FOR INSERT
  WITH CHECK (
    EXISTS (SELECT 1 FROM profiles WHERE id = auth.uid() AND role = 'admin')
  );
```

10. Deployment Architecture

```
GitHub (main branch)
├──
├── ── Auto-deploys → Vercel
├── ── Next.js 15 Web App
├── ── Global CDN
```

```

Environment: NEXT_PUBLIC_API_URL
NEXT_PUBLIC_SUPABASE_URL
NEXT_PUBLIC_SUPABASE_ANON_KEY

└─ Manual deploy → Railway
Express Backend (Docker container)
Auto-restart on crash
Environment: SUPABASE_SERVICE_ROLE_KEY
GEMINI_API_KEY
UPLOADTHING_SECRET
DATABASE_URL
JWT_SECRET

Supabase (managed):
- PostgreSQL 15 database
- 44 applied migrations
- pgvector extension enabled
- Auth configuration
- Storage bucket (for note images)

```

The environment variable split is critical:

- `NEXT_PUBLIC_*` variables are bundled into the client-side JavaScript by Next.js. Only safe, public values go here: the Supabase URL, the anon key (safe — RLS enforces access control), the API URL.
- Non-prefixed variables on Vercel stay server-side (for Server Components only)
- The service role key exists ONLY on Railway — never on Vercel in any form

The API Proxy

Next.js rewrites proxy API calls through the frontend domain, avoiding CORS issues:

```

// next.config.ts
async rewrites() {
  return [
    {
      source: '/api/:path*',
      destination: `${process.env.NEXT_PUBLIC_API_URL}/api/:path*`
    }
  ];
}

```

This means the frontend calls `/api/notes` and Next.js proxies to `https://railway-backend.up.railway.app/api/notes` . The browser only ever sees calls to the Vercel domain. The actual backend URL is never exposed to clients.

11. The Scale of the System

Metric	Count
Database migrations	44 SQL files
Backend service domains	7
API endpoints	50+
Frontend routes	15+
Plate.js editor plugins	20+
AI systems	3 (Editor, Note Tools, Library Assistant)
AI model providers supported	4 (OpenAI, Anthropic, Groq, Google)
i18n message keys per language	200+
Client applications	3 (Web, Chrome Extension, GNOME Extension)
Deployment environments	3 (Vercel, Railway, Supabase)
Lines of TypeScript (estimate)	15,000+

12. What the Six Sections That Follow Are About

Each following presenter explains the subsystem they built. This table maps each section to the architectural decisions that govern it:

Section	Domain	Core Architectural Decisions
2 — Auth	JWT, sessions, profiles	Two Supabase clients, <code>getUser()</code> vs <code>getSession()</code> verification, httpOnly cookies, auto-refresh middleware, database trigger for profile creation
3 — Notes	Editor, sharing, organization	Plate.js JSON AST, <code>content_text</code> extraction for AI, sharing roles, auto-save debounce, public note

		publishing
4 — AI	RAG, embeddings, streaming	<code>text-embedding-004</code> 768-dim vectors, pgvector cosine search, three-layer AI architecture, Vercel AI SDK streaming, multi-model support
5 — Data Catalog	Hierarchical course catalog	URL-driven cascading filter state, SSR with 5-minute cache revalidation, admin-only writes, bilingual schema (<code>name_en</code> / <code>name_ar</code>)
6 — Daily Planning	Tasks, habits, Pomodoro, social	JSONB task storage, streak calculation with SQL CTEs and window functions, <code>get_friends_leaderboard</code> RPC function, <code>syncLogEmbedding</code> async pipeline
7 — Admin, i18n, Extensions, Deployment	All infrastructure	RTL-first CSS logical properties, <code>declarativeNetRequest</code> blocking, GNOME Shell GJS APIs, Vercel + Railway environment variable split